VERİ BİLİMCİ MÜFREDATI

03 — NumPy

Ayrıntılı Konu Anlatımı

Bu modülün amacı

Python listesinden başlayıp çok boyutlu sayısal dizilerle düşünmeye geçmek; shape, dtype, indexing, broadcasting ve vectorization kavramlarını gerçekten anlamak; istatistik ve lineer cebir işlemlerini NumPy ile güvenli ve hızlı biçimde uygulamak.

Öğrenme çıktısı	Beklenen seviye
ndarray yapısını ve Python listelerinden farkını açıklamak	Temel
shape, ndim, size ve dtype kavramlarını kullanmak	Temel
indexing, slicing ve boolean masking yapmak	Orta
broadcasting kurallarını yorumlamak	Orta
vektörize kod yazmak ve döngüleri azaltmak	Orta
aggregation, random ve linalg işlemlerini kullanmak	Orta
veri bilimi problemlerinde NumPy tabanını görmek	Uygulamalı

1. NumPy nedir ve neden kullanılır?

NumPy (Numerical Python), Python ekosisteminde sayısal hesaplamanın temel kütüphanesidir. Pandas, SciPy, scikit-learn ve birçok makine öğrenmesi kütüphanesi NumPy dizilerini doğrudan veya dolaylı olarak kullanır. NumPy'nin merkezindeki veri yapısı ndarray'dir: aynı veri tipindeki elemanları, çok boyutlu ve bellekte verimli bir yapıda tutar.

Python listesi	NumPy ndarray
Farklı veri tiplerini aynı yapıda tutabilir	Genellikle tek bir dtype kullanır
Eleman bazlı matematik için döngü gerekir	Vektörize işlemler doğal ve hızlıdır
Bellek maliyeti daha yüksektir	Sayısal veride daha kompakt olabilir
Genel amaçlı konteynerdir	Sayısal hesaplama için optimize edilmiştir

```
import numpy as np

prices = np.array([120.0, 85.5, 199.9, 42.0])
print(prices)
print(type(prices))
```

Data Scientist bakışı
NumPy'yi ezberlenecek fonksiyonlar kümesi olarak değil, "veriyi matris/dizi olarak düşünme" altyapısı olarak gör. ML modelleri nihayetinde çoğu zaman X feature matrisi ve y hedef vektörü üzerinde çalışır.

2. ndarray'in anatomisi: shape, ndim, size, dtype

Bir ndarray'i anlamak için ilk bakacağın dört özellik shape, ndim, size ve dtype olmalıdır.

```
x = np.array([[10, 20, 30],
              [40, 50, 60]])

print(x.shape)   # (2, 3)
print(x.ndim)    # 2
print(x.size)    # 6
print(x.dtype)   # örn. int64
```

Özellik	Anlamı	Örnek
shape	Her eksendeki uzunluk	(2, 3)
ndim	Boyut/eksen sayısı	2
size	Toplam eleman sayısı	6
dtype	Elemanların veri tipi	int64

Shape kavramı özellikle makine öğrenmesinde kritiktir. Genel gösterim X.shape = (n_samples, n_features) şeklindedir. Örneğin 10.000 müşteriden 25 feature topladıysan X.shape yaklaşık (10000, 25) olur.

3. Array oluşturma yöntemleri

```
np.array([1, 2, 3])
np.zeros((3, 4))
np.ones((2, 5))
np.full((2, 3), 7)
np.arange(0, 10, 2)
np.linspace(0, 1, 6)
np.eye(4)
np.empty((2, 2))
```

Fonksiyon	Ne zaman kullanılır?
zeros	Başlangıç matrisi / placeholder
ones	Sabit 1 matrisi
full	Tek bir sabit değerle doldurmak
arange	Adımlı tam sayı / düzenli aralık
linspace	Belirli aralıkta eşit aralıklı n nokta
eye	Birim matris
empty	İçeriği başlatmadan hızlı allocation

Dikkat
np.empty sıfırlarla dolu garanti etmez. Bellekte o anda bulunan değerleri gösterebilir.

4. dtype ve veri tipi dönüşümü

NumPy dizileri tek bir dtype etrafında optimize edilir. Bir dizide float bulunması, integer değerlerin de float'a yükseltilmesine yol açabilir.

```
a = np.array([1, 2, 3], dtype=np.float32)
print(a.dtype)

b = a.astype(np.int32)
print(b)
```

- int8/int16/int32/int64: tam sayılar için farklı bellek genişlikleri.
- float32/float64: kayan noktalı sayılar; ML tarafında float32 sık görülür.
- bool: maskeleme ve koşullu seçim için çok önemlidir.
- astype dönüşüm yapar; özellikle float -> int dönüşümünde ondalık bölümün kaybolacağını unutma.

5. Indexing: eleman seçme

```
a = np.array([10, 20, 30, 40, 50])
print(a[0])     # 10
print(a[-1])    # 50

m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m[0, 1])  # 2
print(m[1, 2])  # 6
```

İki boyutlu dizilerde m[row, column] yazımı kullanılır. Çok boyutlu dizilerde virgülle ayrılmış her indeks bir ekseni temsil eder.

6. Slicing: alt dizi seçme

```
a = np.array([10, 20, 30, 40, 50, 60])
print(a[1:4])    # [20 30 40]
print(a[:3])     # [10 20 30]
print(a[::2])    # [10 30 50]

m = np.arange(1, 13).reshape(3, 4)
print(m[:, 1])       # tüm satırlar, 2. sütun
print(m[0:2, 1:3])   # alt matris
```

> **Önemli fark**
> NumPy slicing çoğu zaman view döndürür; yani alt dizide yaptığın değişiklik ana diziyi etkileyebilir. Bağımsız kopya gerekiyorsa .copy() kullan.

```
x = np.array([1, 2, 3, 4])
y = x[1:3]
y[0] = 999
print(x)  # [  1 999   3   4]

z = x[1:3].copy()
```

7. Boolean masking ve koşullu seçim

Veri biliminde en sık kullanılan NumPy desenlerinden biridir. Bir koşul üretirsin; sonuç True/False maskesidir. Bu maskeyle istediğin gözlemleri seçersin.

```
sales = np.array([120, 450, 80, 900, 310])
mask = sales > 300
print(mask)
print(sales[mask])

# Tek satırda
high_sales = sales[sales > 300]


ages = np.array([19, 32, 44, 17, 28])
incomes = np.array([12000, 45000, 72000, 5000, 33000])

selected = incomes[(ages >= 25) & (incomes > 30000)]
```

Kural
NumPy koşullarını birleştirirken Python'daki and/or yerine &, | ve ~ kullanılır. Her koşulu paranteze almak güvenlidir.

8. Fancy indexing

```
a = np.array([10, 20, 30, 40, 50])
print(a[[0, 2, 4]])

m = np.array([[10, 11],
              [20, 21],
              [30, 31]])
print(m[[0, 2]])
```

Fancy indexing, belirli indeks listesini kullanarak seçme yapar. Slicing'den farklı olarak genellikle kopya üretir.

9. Reshape, flatten, ravel ve transpose

```
a = np.arange(12)
m = a.reshape(3, 4)

print(m.T)
print(m.flatten())
print(m.ravel())
```

İşlem	Amaç	Not
reshape	Shape değiştirme	Toplam eleman sayısı korunmalı
T / transpose	Eksenleri yer değiştirme	2D'de satır-sütun değişir
flatten	1D kopya üretme	Bağımsız kopya
ravel	Mümkünse view döndürme	Daha bellek dostu olabilir

10. Eksen (axis) mantığı

NumPy'de axis parametresi yeni başlayanların en çok karıştırdığı konulardan biridir. 2D matris için axis=0 satır eksenini azaltır ve sütun bazlı sonuç üretir; axis=1 sütun eksenini azaltır ve satır bazlı sonuç üretir.

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])

print(x.sum())         # 21
print(x.sum(axis=0))   # [5 7 9]  sütun toplamları
print(x.sum(axis=1))   # [ 6 15]  satır toplamları
```

Hatırlama yöntemi

axis=0: 0 numaralı eksen yok edilir; satırlar boyunca toplanır ve sütun başına sonuç kalır. axis=1: sütun ekseni yok edilir; satır başına sonuç kalır.

11. Element-wise işlemler

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)
print(a - b)
print(a * b)
print(b / a)
print(a ** 2)
print(np.sqrt(b))
print(np.log(b))
```

Buradaki * matris çarpımı değil, eleman bazlı çarpımdır. Matris çarpımı için @ veya np.matmul kullanılır.

12. Broadcasting

Broadcasting, farklı shape'lerdeki dizilerin belirli kurallara uyuyorsa açıkça kopyalanmadan birlikte işlem yapabilmesini sağlar. Bu, NumPy'nin en güçlü kavramlarından biridir.

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])

print(x + 10)               # scalar broadcasting

bias = np.array([100, 200, 300])
print(x + bias)             # (2,3) + (3,)
```

Broadcasting uyumluluğu sağdan sola kontrol edilir. Karşılaştırılan boyutlar eşitse veya boyutlardan biri 1 ise uyumludur.

Shape A	Shape B	Uyumlu mu?	Sonuç
(2, 3)	(3,)	Evet	(2, 3)
(4, 1)	(1, 5)	Evet	(4, 5)
(10, 3)	(10,)	Hayır	ShapeError

(8, 1, 6)	(7, 6)	Evet	(8, 7, 6)

Pratik sezgi
Bir feature sütununu standardize ederken (X - mean) / std işlemi broadcasting sayesinde satır satır otomatik uygulanır.

13. Vectorization: neden döngü yerine NumPy?

Vectorization, işlemi Python for döngüsü yerine NumPy'nin optimize edilmiş düşük seviyeli rutinlerine bırakmaktır. Amaç yalnızca daha kısa kod değil; daha yüksek performans ve daha açık matematiksel ifade sağlamaktır.

```
# Döngülü yaklaşım
result = []
for value in prices:
    result.append(value * 1.20)

# Vektörize yaklaşım
result = prices * 1.20
```

- Büyük dizilerde Python döngülerinden çoğu zaman belirgin biçimde hızlıdır.
- Matematiksel formülü doğrudan kodda ifade eder.
- Broadcasting ile birlikte güçlü hale gelir.
- Ancak gereksiz dev ara diziler üretmek bellek maliyetini artırabilir.

14. Aggregation ve özet istatistikler

```
x = np.array([4, 7, 10, 13, 16])

x.sum()
x.mean()
np.median(x)
x.min()
x.max()
x.std()
x.var()
x.argmin()
x.argmax()
np.percentile(x, [25, 50, 75])
```

Veri analizinde ortalama tek başına yeterli değildir. Median, percentile, std ve min/max ile dağılım hakkında daha sağlam bir resim elde edilir.

15. NaN ve eksik değerlerle çalışma

```
x = np.array([10.0, np.nan, 30.0, 40.0])

print(np.isnan(x))
print(np.mean(x))       # nan
print(np.nanmean(x))    # 26.666...
print(np.nansum(x))
print(np.nanmedian(x))
```

Dikkat
np.nan kayan nokta dünyasına aittir. Integer array içine doğrudan NaN koymak dtype dönüşümü gerektirebilir.

16. Sıralama, benzersiz değerler ve arama

```
x = np.array([5, 2, 8, 2, 9, 5, 2])

np.sort(x)
np.argsort(x)
np.unique(x)
np.unique(x, return_counts=True)
np.where(x > 5)
np.clip(x, 3, 7)
```

np.unique kategorik değerlerin frekansını hızlı görmek için; np.where koşula göre indeks/sonuç üretmek için; np.clip uç değerleri belirli sınırlar içine çekmek için kullanışlıdır.

17. Birleştirme ve bölme işlemleri

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.concatenate([a, b], axis=0)
np.concatenate([a, b], axis=1)
np.vstack([a, b])
np.hstack([a, b])

np.split(np.arange(8), 4)
```

Shape kontrolü
concatenate kullanırken birleştirme yaptığın axis dışındaki boyutların eşleşmesi gerekir.

18. Random modülü ve tekrarlanabilirlik

Random sayı üretimi train-test örneklemesi, bootstrap, simülasyon, ağırlık başlatma ve deneysel çalışmalar için kullanılır. Modern NumPy yaklaşımı Generator nesnesini kullanmaktır.

```
rng = np.random.default_rng(42)

rng.integers(0, 10, size=5)
rng.random(5)
```

```
rng.normal(loc=0, scale=1, size=1000)
rng.choice([0, 1], size=20, p=[0.7, 0.3])

idx = rng.permutation(100)
```

Reproducibility
Aynı seed aynı pseudo-random akışın tekrar üretilebilmesini sağlar. Portföy ve deney kodlarında seed yönetimini bilinçli yap.

19. İstatistiksel simülasyon örneği

```
rng = np.random.default_rng(123)

# 10.000 kullanıcının dönüşümü: p = 0.08
converted = rng.binomial(n=1, p=0.08, size=10_000)
print(converted.mean())

# Bootstrap ile ortalama dağılımı
boot_means = np.array([
    rng.choice(converted, size=len(converted), replace=True).mean()
    for _ in range(1000)
])

print(np.percentile(boot_means, [2.5, 97.5]))
```

Bu örnek A/B testing ve istatistik modüllerine doğrudan bağlanır: NumPy, teorik kavramları simülasyonla görselleştirmenin en pratik araçlarından biridir.

20. Lineer cebir: vektör ve matris işlemleri

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a @ b)            # dot product
print(np.dot(a, b))

A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

print(A @ B)
```

İşlem	NumPy
Dot product	a @ b veya np.dot(a,b)
Matrix multiplication	A @ B veya np.matmul(A,B)
Transpose	A.T
Determinant	np.linalg.det(A)
Inverse	np.linalg.inv(A)
Linear system Ax=b	np.linalg.solve(A,b)
Eigen decomposition	np.linalg.eig(A)
Norm	np.linalg.norm(a)

21. Ax = b çözme ve inverse neden her zaman tercih edilmez?

```
A = np.array([[3.0, 1.0],
              [1.0, 2.0]])
b = np.array([9.0, 8.0])

x = np.linalg.solve(A, b)
print(x)
print(A @ x)
```

Lineer sistem çözmek için doğrudan inverse hesaplayıp inv(A) @ b yapmak yerine np.linalg.solve kullanmak genellikle daha kararlı ve daha verimlidir.

22. Standardizasyonu NumPy ile elle yapmak

```
X = np.array([
    [20, 1200, 3],
    [35, 2400, 7],
    [50, 1800, 5]
], dtype=float)

mu = X.mean(axis=0)
sigma = X.std(axis=0)
X_scaled = (X - mu) / sigma

print(X_scaled)
```

Burada mu ve sigma shape olarak (3,) olur. X shape (3,3) olmasına rağmen broadcasting sayesinde her feature kendi ortalama ve standart sapmasıyla ölçeklenir.

23. np.where, select ve parça bazlı dönüşümler

```
sales = np.array([120, 600, 250, 900])
label = np.where(sales >= 500, 'high', 'low')

scores = np.array([25, 55, 72, 91])
conditions = [scores < 50, scores < 75, scores >= 75]
choices = ['low', 'medium', 'high']
category = np.select(conditions, choices)
```

Bu yaklaşım feature engineering sırasında kategorik feature üretmek için kullanılabilir; fakat karmaşık iş kurallarında okunabilirlik her zaman önceliklidir.

24. View vs copy: sessiz bug kaynağı

NumPy performans için sık sık veriyi kopyalamak yerine aynı belleğe bakan view'lar üretir. Bu davranış bilinmezse bir ara değişkeni değiştirdiğinde orijinal array istemeden değişebilir.

```
x = np.arange(6)
a = x[1:4]        # view olma ihtimali yüksek
b = x[[1, 2, 3]]  # fancy indexing: kopya

print(np.shares_memory(x, a))
print(np.shares_memory(x, b))
```

25. Bellek düzeni ve performans sezgisi

NumPy dizileri bellekte çoğunlukla C-order (satır-major) veya Fortran-order (sütun-major) düzeninde tutulabilir. Çoğu günlük veri bilimi görevinde bunu elle yönetmezsin, fakat büyük matrislerde contiguous bellek, view/kopya davranışı ve dtype seçimi performansı etkileyebilir.

- float32, float64'ün yarısı kadar bellek kullanır; ancak hassasiyeti daha düşüktür.
- Gereksiz astype ve copy çağrıları büyük veride pahalıdır.
- Vektörizasyon her zaman az bellek demek değildir; ara array'leri izle.
- Önce doğru ve okunabilir kod, sonra ölçüm/profiling ile optimizasyon.

26. NumPy ve Pandas ilişkisi

Pandas; index, sütun isimleri, heterojen veri tipleri ve eksik veri semantiği gibi üst seviye özellikler sunar. NumPy ise altında daha yalın ve hızlı sayısal array modelini sağlar. Pandas'taki bir DataFrame'in birçok sayısal işlemi NumPy dünyasına dayanır.

İhtiyaç	Tercih
Etiketli tablo, sütun isimleri, join/groupby	Pandas
Saf sayısal matris hesapları	NumPy
ML modeline feature matrix verme	NumPy/Pandas ikisi de olabilir
Lineer cebir / simülasyon	NumPy
Zaman serisi indexleme / kategorik kolonlar	Pandas

27. Gerçek vaka: E-Commerce sepet analizi

Bir e-ticaret şirketinde 8 sipariş için sepet tutarı ve ürün adedi verildiğini düşün. Amaç: sipariş başına fiyat, yüksek değerli siparişler, normalize skor ve basit segment üretmek.

```
import numpy as np

revenue = np.array([120, 450, 80, 900, 310, 1250, 640, 150], dtype=float)
items   = np.array([2,   5,   1,  7,   3,   8,    4,   2], dtype=float)

avg_item_price = revenue / items
high_value = revenue >= 500

z = (revenue - revenue.mean()) / revenue.std()
segment = np.where(z >= 1, 'VIP', np.where(z >= 0, 'Medium', 'Low'))

print(avg_item_price)
print(revenue[high_value])
print(segment)
```

Vaka soruları

1. Revenue ortalaması, medianı, standart sapması ve 90. percentilini hesapla.
2. 500 TL üzerindeki siparişlerin toplam gelirin yüzde kaçını oluşturduğunu bul.
3. Ürün başına ortalama fiyatı 150 TL üzerinde olan siparişleri seç.
4. Revenue ve item count'u iki sütunlu feature matrisi haline getir.
5. Her feature'ı kendi ortalaması ve std'si ile standardize et.
6. Bir seed kullanarak sipariş indekslerini karıştır ve ilk %75'i train, kalanını test olarak ayır.

28. Mini proje: Customer Feature Matrix Builder

Bu mini proje, NumPy bilgisini ML'e bağlar. 5.000 müşterilik sentetik veri üret; yaş, gelir, son 30 gün sipariş adedi, toplam harcama ve web oturum sayısı feature'larını oluştur.

7. np.random.default_rng ile veriyi reproducible üret.
8. Feature'ları np.column_stack ile tek X matrisinde birleştir.
9. Shape, dtype ve eksik/uygunsuz değer kontrolleri yap.
10. Gelir ve harcama feature'larına uç değer ekle; percentile ile incele.
11. Boolean masking ile aktif ve yüksek değerli müşteri segmentleri üret.
12. X matrisini sütun bazında standardize et.
13. Correlation matrix için np.corrcoef kullan ve en yüksek ilişkileri yorumla.
14. Train/test indekslerini permutation ile ayır.
15. Çıktıları kısa bir README içinde açıklayarak GitHub'a koy.

```
rng = np.random.default_rng(42)
n = 5000

age = rng.integers(18, 75, n)
income = rng.lognormal(mean=10.4, sigma=0.45, size=n)
orders = rng.poisson(lam=3.2, size=n)
spend = orders * rng.gamma(shape=2.0, scale=180.0, size=n)
sessions = rng.poisson(lam=9, size=n)

X = np.column_stack([age, income, orders, spend, sessions])
print(X.shape)
```

29. Sık yapılan hatalar

Hata	Neden problem?	Doğru yaklaşım
Python and/or kullanmak	Array truth value belirsizdir	&, |, ~ ve parantez kullan
shape kontrol etmeden işlem yapmak	Broadcasting sessizce yanlış sonuç üretebilir	Her adımda shape düşün
View davranışını bilmemek	Orijinal veri yanlışlıkla değişir	Gerekirse .copy() kullan
for döngüsüne erken başvurmak	Yavaş ve gereksiz uzun kod	Önce vectorization ara
np.mean ile NaN içeren array	Sonuç nan olabilir	nanmean/nanmedian veya temizleme
float karşılaştırmasını == ile yapmak	Hassasiyet problemi	np.isclose / np.allclose
Matrix multiply için * kullanmak	Element-wise çarpım yapar	@ kullan

30. Mülakat tipi sorular

16. Python listesi ile NumPy array arasındaki temel farklar nelerdir?
17. Broadcasting nedir? (10, 3) ile (3,) neden toplanabilir?
18. axis=0 ve axis=1 ne anlama gelir?
19. View ve copy arasındaki fark nedir?
20. Boolean masking nasıl çalışır?
21. np.reshape hangi durumda hata verir?
22. * ile @ arasındaki fark nedir?
23. NaN içeren bir array'in ortalamasını nasıl güvenli hesaplarsın?
24. Vektörizasyon neden genellikle for döngüsünden hızlıdır?
25. Bir feature matrix'i sütun bazında nasıl standardize edersin?
26. np.random.seed yaklaşımı ile default_rng arasındaki kavramsal fark nedir?
27. np.linalg.solve neden inverse hesaplamaya tercih edilir?

31. Pratik alıştırmalar

28. 1'den 100'e kadar sayıları içeren array üret; çift sayıları boolean mask ile seç.
29. 1..24 değerlerini (4,6) matrise çevir; ilk iki satır ve son üç sütunu seç.
30. Her sütunun mean ve std'sini hesapla; matrisi standardize et.
31. 1000 normal dağılım örneği üret; 5, 50 ve 95 percentile'leri hesapla.
32. İki vektörün dot product'ını hem @ hem np.dot ile doğrula.
33. (100, 5) boyutlu sentetik X üret; her satırın maksimum feature değerini hesapla.
34. Bir array'de negatif değerleri 0'a clip et.
35. 10.000 Bernoulli denemesi üret ve deneysel dönüşüm oranını hesapla.
36. np.unique ile kategori frekanslarını çıkar.
37. Bir slicing view oluştur; değişikliğin orijinale etkisini gözlemle ve copy ile karşılaştır.

32. NumPy için final kontrol listesi

Kontrol	Kendine sor

Array zihniyeti	Liste yerine shape ve axis ile düşünebiliyor muyum?
Indexing	1D/2D slicing ve masking yapabiliyor muyum?
Broadcasting	İşlem öncesi shape uyumluluğunu açıklayabiliyor muyum?
Vectorization	Basit döngüleri array işlemlerine çevirebiliyor muyum?
Statistics	mean/std/percentile/NaN-safe fonksiyonları biliyor muyum?
Random	default_rng + seed ile reproducible simülasyon yapabiliyor muyum?
Linear algebra	dot, @, transpose, solve, norm kavramlarını kullanabiliyor muyum?
Debugging	dtype, shape, view/copy hatalarını teşhis edebiliyor muyum?

Bu modülü bitirmiş sayılmak için

Kodlara bakmadan bir 2D array oluşturup filtreleyebilmeli, reshape yapabilmeli, sütun bazlı aggregation uygulayabilmeli, broadcasting ile standardize edebilmeli ve basit lineer cebir işlemlerini açıklayabilmelisin.